

```

%%writefile vector_add.cu
#include <iostream>
#include <cuda.h>

using namespace std;

// CUDA Kernel
__global__ void vectorAdd(float *A, float *B, float *C, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        C[i] = A[i] + B[i];
    }
}

int main() {
    // Hardcoded for 1 million elements
    int n = 1000000;
    size_t size = n * sizeof(float);

    // Allocate memory on the Host (CPU)
    float *h_A = (float *)malloc(size);
    float *h_B = (float *)malloc(size);
    float *h_C = (float *)malloc(size);

    // Initialize vectors with hardcoded data
    for (int i = 0; i < n; i++) {
        h_A[i] = (float)(i % 100); // 0,1,2...99,0,1,2...
        h_B[i] = (float)((i % 100) * 2); // 0,2,4...198,0,2...
    }

    // Allocate memory on the Device (GPU)
    float *d_A, *d_B, *d_C;
    cudaMalloc((void **)&d_A, size);
    cudaMalloc((void **)&d_B, size);
    cudaMalloc((void **)&d_C, size);

    // Copy data to GPU
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (n + threadsPerBlock - 1) / threadsPerBlock;

    // Timer
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    // Launch Kernel

```

```

    cudaEventRecord(start);
    vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, n);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    float ms = 0;
    cudaEventElapsedTime(&ms, start, stop);

    // Copy result back to CPU
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    // Verify first 5 results
    cout << "--- First 5 Results ---\n";
    for (int i = 0; i < 5; i++) {
        cout << h_A[i] << " + " << h_B[i] << " = " << h_C[i] << "\n";
    }
    cout << "... \n[Success] Processed " << n << " elements.\n";
    cout << "[TIME] GPU Execution Time: " << ms << " ms\n";

    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    free(h_A); free(h_B); free(h_C);

    return 0;
}

```

Overwriting vector_add.cu

```

!nvcc vector_add.cu -o vector_add
!./vector_add

```

```

nvcc warning : Support for offline compilation for architectures prior to '<comp
--- First 5 Results ---
0 + 0 = 0
1 + 2 = 3
2 + 4 = 6
3 + 6 = 9
4 + 8 = 12
...
[Success] Processed 1000000 elements.
[TIME] GPU Execution Time: 0.266144 ms

```

```

%%writefile vector_add_demo.cu
#include <iostream>
#include <cuda.h>
using namespace std;

__global__ void vectorAdd(int *A, int *B, int *C) {
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}

```

```

int main() {
    int n = 5;
    int h_A[] = {1, 2, 3, 4, 5};
    int h_B[] = {10, 20, 30, 40, 50};
    int h_C[5];

    int *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, n * sizeof(int));
    cudaMalloc(&d_B, n * sizeof(int));
    cudaMalloc(&d_C, n * sizeof(int));

    cudaMemcpy(d_A, h_A, n * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, n * sizeof(int), cudaMemcpyHostToDevice);

    vectorAdd<<<1, n>>>>(d_A, d_B, d_C);

    cudaMemcpy(h_C, d_C, n * sizeof(int), cudaMemcpyDeviceToHost);

    cout << "--- Hardcoded Demo: Vector Addition ---\n";
    for (int i = 0; i < n; i++)
        cout << h_A[i] << " + " << h_B[i] << " = " << h_C[i] << "\n";

    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    return 0;
}

```

Writing vector_add_demo.cu

```

!nvcc vector_add_demo.cu -o vector_add_demo
!./vector_add_demo

```

```

nvcc warning : Support for offline compilation for architectures prior to '<comp
--- Hardcoded Demo: Vector Addition ---
1 + 10 = 11
2 + 20 = 22
3 + 30 = 33
4 + 40 = 44
5 + 50 = 55

```

Start coding or [generate](#) with AI.

